

PerpScope

A Perpetual-DEX Analytics Platform

Project Report & Technical Walkthrough

PerpScope turns raw perpetual-futures exchange data into three concrete trading decisions. It combines a self-built daily data pipeline, a trained machine-learning market-regime classifier, and live cross-exchange calculators into a single React dashboard.

Three features: (1) Market Analytics — an end-to-end ELT pipeline (Python → BigQuery → dbt) feeding ten charts, plus an embedded Hidden Markov Model that classifies Bitcoin's market regime; (2) Funding Rate Arbitrage Scanner — a cross-exchange delta-neutral carry screener; (3) DEX Trading Cost Comparison — a live order-book execution-cost calculator.

Companion document: *an accompanying build manual ("Market Analytics ETL — Build-It-Yourself Manual", attached separately as a PDF) documents the pipeline phase-by-phase with full SQL. This report is the higher-level walkthrough: what each feature does, the logic behind it, and why it produces a usable trading edge.*

0 System Overview

PerpScope is a React + TypeScript single-page app backed by a cloud data pipeline. The three features draw on data at different cadences, so each uses the serving pattern that fits it:

Feature	Data cadence	How it is served
Market Analytics	Daily snapshot	Pre-computed once per day by a cloud pipeline, exported to static JSON the browser fetches.
Funding Arb Scanner	Live (1–20 min)	Fetches live in the browser from three exchange APIs, computed client-side.
DEX Cost Comparison	Live (on demand)	Pulls live order books per trade request, computes cost client-side.

Design philosophy (applies to all three): push each piece of logic to where it is cheapest and most auditable — SQL inside the warehouse for anything that can be a query, exported model parameters for anything that must run instantly in the browser, and a once-a-day cron rather than per-visit API calls for data that only changes daily. The result is a system that is cheap to run (entirely on free tiers), reproducible, and fast for the end user.

3 Exchange / data APIs	Python extract	BigQuery + dbt	JSON / live compute	React dashboard
------------------------	----------------	----------------	---------------------	-----------------

1 Feature 1 — Market Analytics

The flagship feature and the largest engineering effort. It is a top-down market cockpit built from three linked dashboards, all fed by a daily ELT (Extract-Load-Transform) pipeline the project builds from scratch rather than reading from a single third-party endpoint.

Dashboard	Question it answers	Key charts
1 · Market Structure	Is the market risk-on or risk-off?	BTC regime (HMM), Fear & Greed, hottest coins, strongest coins
2 · Open Interest & Capital Rotation	Where is leverage concentrated and moving?	OI-dominance treemap, 7-day OI movers, OI/market-cap leverage bar
3 · Smart-Money Analytics	What are the best wallets doing right now?	Smart-money leaderboard, net positioning by asset, 24h capital flows

Why it matters: collapses ten separate signals — regime, sentiment, momentum, leverage concentration, leverage rotation, smart-money consensus and flow — into one screen. A trader forms a full top-down view (macro regime → capital rotation → smart-money confirmation) in one sitting instead of stitching together ten browser tabs and several paid terminals.

1.1 The ELT Data Pipeline

Extract-Load-Transform, not the older Transform-first ETL: raw API responses are landed in the BigQuery warehouse unmodified first, and every cleaning/business-logic step then happens as version-controlled SQL inside the warehouse (via dbt). This is the modern analytics-engineering pattern — it means a bad transform can be fixed and the whole history rebuilt from raw without ever re-hitting a rate-limited API.

1 · Cloud setup	2 · Extract (Python)	3 · Load (raw)	4 · Transform (dbt)	5 · Orchestrate	6 · Serve
-----------------	----------------------	----------------	---------------------	-----------------	-----------

Phase	What happens	Tooling & detail
1 · Cloud setup	Provision an empty, ready warehouse.	Google BigQuery project on the free sandbox tier; a service account with BigQuery Data Editor + Job User roles; three datasets — raw / staging / marts.
2 · Extract	Pull today's data from every source.	Python (requests / pandas). Each source is an isolated extractor returning flat records tagged with a snapshot_date. Sources below.
3 · Load	Land raw records in BigQuery.	google-cloud-bigquery client. WRITE_TRUNCATE per snapshot_date partition = idempotent: re-running the same day overwrites, never duplicates. Adds a _loaded_at timestamp.
4 · Transform	Clean & reshape into fact tables.	dbt Core (SQL). Three layers: raw → staging (typed, deduped views) → marts (6 reusable fact tables + 1 dimension). Rebuilt each run via CREATE OR REPLACE TABLE.
5 · Orchestrate	Run the whole thing daily, unattended.	GitHub Actions cron (06:00 UTC) + a manual trigger. Service-account key injected from an encrypted secret; the refreshed JSON export is auto-committed back to the repo.
6 · Serve	Deliver data to the app.	Static JSON export (public/market/*.json) fetched client-side by React — no browser-exposed warehouse credentials. Power BI can connect live via the native BigQuery connector.

Data sources

Source	Provides	Feeds
Hyperliquid — metaAndAssetCtxs	Per-asset open interest, 24h volume, funding, mark price	Hottest coins, OI dominance, OI/mcap, movers

Source	Provides	Feeds
Hyperliquid — candleSnapshot	Daily OHLCV per asset	Strongest coins, 7-day price change, BTC regime
Hyperliquid — leaderboard	~40,000 wallet addresses + windowed PnL/volume	Smart-money cohort selection
Hyperliquid — clearinghouseState	One wallet's live open positions	Net positioning, capital flows, top holdings
Hyperliquid — portfolio	One wallet's daily equity curve	Risk-adjusted smart-money scoring (Sharpe etc.)
CoinGecko — coins/markets	Spot market cap, symbol, rank	OI/mcap ratio, top-by-mcap universe, symbol map
Alternative.me — fng	Fear & Greed index history	Sentiment gauge + 90-day chart

Production-grade detail worth noting: the CoinGecko free tier can return an HTTP 200 with a silently truncated/degraded coin list when rate-limited. The extractor guards against this with exponential-backoff retries plus a sanity check that page 1 must contain BTC and ETH, aborting the run rather than loading corrupted market caps. Endpoint shapes are also eyeballed live before wiring in, since the Hyperliquid leaderboard host has changed before.

1.2 Data Architecture — raw → staging → marts

Three BigQuery datasets, three purposes — the heart of the analytics-engineering design:

- **perpscope_raw** — the landing zone. API responses stored exactly as received, partitioned by snapshot_date. The audit trail; lets transforms be re-run without re-hitting APIs.
- **perpscope_staging** — one view per raw table, 1:1 with the source: cast types, rename to snake_case, dedupe to the freshest load per day. No joins, no business logic.
- **perpscope_marts** — the analytics layer: clean, reusable fact & dimension tables where business logic lives, materialized as physical tables for fast reads.

Core design principle — the warehouse stores facts, the client sorts and slices

Each fact table holds EVERY row at its grain (all assets, all days) plus every column a chart might sort or filter on. No rank, no LIMIT, no is_latest, no top-N in the warehouse. React / Power BI apply ORDER BY / LIMIT / “top 5” themselves.

Only ONE definitional filter lives in-warehouse: which wallets are the smart-money cohort.

Consequence: ten charts are served by just six fact tables + one dimension. A new chart wanting “top 20” instead of “top 5” — or a watchlist view — needs zero schema changes.

The six fact tables

Fact table	Grain	What it stores
fct_asset_metrics_daily	coin × day	The wide per-asset fact powering 5 charts: 24h volume change, OI dominance, OI change (24h/7d), price change, OI/mcap ratio, 30-day relative strength vs BTC.
fct_fear_greed	date	Full Fear & Greed index history (value + classification).
fct_btc_regime	day (BTC)	HMM regime label + confidence per day (see 1.3).
fct_asset_positioning_daily	coin × day	Per-coin cohort long/short/flat counts, net notional, and day-over-day capital inflow/outflow.
fct_trader_daily	wallet × day	Smart-money cohort stats: PnL, ROI, Sharpe, profit factor, drawdown, and the composite smart score.

Fact table	Grain	What it stores
fct_trader_positions	position	Each cohort wallet's individual open positions (coin, side, size, unrealized PnL).

Change metrics use SQL window functions: `LAG(... ) OVER (PARTITION BY coin ORDER BY snapshot_date)` looks back 1, 7 and 30 days per coin in a single pass to compute “vs yesterday” and “vs last week” deltas. Data-quality tests (dbt) assert grain uniqueness, and that oi_dominance sums to 1.0 per day and long/short/flat shares sum to 1.0 per coin.

Why it matters: history accrues correctly. Because facts retain every day (not just today), day-over-day and 7-day metrics compute automatically once enough snapshots exist — and cohort membership on any past day is always recoverable, not overwritten.

1.3 BTC Market-Regime Classifier (Hidden Markov Model)

This is a genuine data-science sub-project, not a single hard-coded rule. The goal: label Bitcoin's current market regime (and the probability of each) from its recent price/funding/volume behaviour, so the whole dashboard has a macro baseline. Because regime detection is unsupervised (there are no ground-truth labels), the approach was to compare model families rigorously and pick the best.

Step 1 — Feature engineering (6 features, four economic axes)

Raw price is never fed to the model — it is not stationary and not comparable across time. Instead six engineered features describe market behaviour:

Feature	Definition	Axis
mom_20	20-day log return	Momentum
trend	close / 50-day SMA – 1	Trend
vol_14	14-day annualized return volatility	Volatility
downside_vol	14-day annualized volatility of negative returns only	Tail / downside risk
funding_ma	7-day mean funding rate	Positioning sentiment
vol_z	30-day volume z-score	Participation

Note on open interest: the free Binance endpoint only serves ~30 days of OI history — too short to train on — so OI is deliberately excluded from the model and the leverage/positioning signal comes from funding + volume instead. The live app, which aggregates OI across exchanges in real time, surfaces OI separately.

Step 2 — Model selection (why a Hidden Markov Model won)

Four model families were trained on ~2,400 days of real BTCUSDT data and compared on multiple axes:

Candidate	Result
K-Means, Agglomerative clustering	Treat each day independently (i.i.d.) — produce choppy regimes that flip every ~10–12 days. No notion of time.
Gaussian Mixture Model (GMM)	Better raw statistical fit, but still i.i.d. and non-persistent. Kept as the fallback model.
Gaussian HMM ← chosen	The only candidate that models time-dependence between days. Produces persistent regimes (~31-day mean duration), the most economically distinct states, and natively exposes transition probabilities.

Evaluation metrics used: cluster validity (silhouette, Davies-Bouldin, Calinski-Harabasz), model selection (BIC / AIC), regime persistence (mean run length), economic distinctness (ANOVA F-test on next-day returns across regimes), and walk-forward time-series cross-validation. The shipped model is a Gaussian HMM with diagonal covariance and 5 states.

The five production regimes: Bull / Risk-on · Accumulation / Recovery · Range / Neutral · Bear / Downtrend · Capitulation / Crash.

Step 3 — Live inference in the browser (no Python at runtime)

The browser cannot run Python / hmmlearn, so the trained parameters (feature scaler + start-probability vector + transition matrix + per-state means & diagonal variances + labels) are exported once from Python to a TypeScript file. The app then re-implements the lightweight inference itself:

Inference (a few matrix operations per day)

Forward algorithm, log-space (numerically stable):

```
log $\alpha$ _t(i) = logsumexp_j[ log $\alpha$ _{t-1}(j) + log A_{ji} ] + log B_i(x_t)
→ today's filtered regime posterior (the confidence %) + next-day transition odds
```

Viterbi decode:

the single most-likely regime path over the whole history (colours the BTC price chart)

B_i(x_t) is the Gaussian emission likelihood of today's standardized feature vector under state i. Because the model runs client-side off live Binance data, the regime updates instantly with no server round-trip.

Why it matters: sets a trader's baseline directional bias before they look at any single asset — trend-follow and hold longer when Bull / Accumulation dominates; take profit faster, size down and favour shorts when Bear / Capitulation gains probability. Because the model exposes the probability of every state, a rising second-place regime is an early warning that a shift is brewing, before price confirms it — something no simple threshold rule can provide.

1.4 Smart-Money Wallet Ranking (2-stage risk-adjusted score)

The naive approach — rank wallets by 30-day PnL — rewards one lucky, over-leveraged bet exactly as highly as genuine, repeatable skill. PerpScope instead scores each wallet on its *risk-adjusted equity curve*, using a two-stage design that keeps the expensive work small.

Stage	Cost	What it does
Stage 1 — prefilter	1 request	Over all ~40,000 leaderboard rows: drop inactive/placeholder accounts (a known ~\$500 sentinel and zero-value/zero-volume wallets), sort by 30-day PnL, keep the top 500 candidates.
Stage 2 — scoring	500 requests	One equity-curve (“portfolio”) call per candidate. Build each wallet's 30-day daily P&L series and compute Sharpe ratio, profit factor, maximum drawdown, win rate and volatility.

Sharpe ratio (30-day) — computed from the equity curve, not the trade fills

```
daily_pnl_t      = cumulative_pnl_t - cumulative_pnl_{t-1}      (isolates trading P&L from deposits)
daily_return_t   = daily_pnl_t / account_value_{t-1}
Sharpe_30d       = ( mean(daily_return) / stdev(daily_return, ddof=1) ) × √365
```

√365 not √252 — crypto perps trade every calendar day. No risk-free rate is subtracted: funding is the closer analogue and is already netted into PnL. Differencing cumulative PnL (rather than account value) removes deposits/withdrawals so the ratio reflects trading skill, not cash movements.

Composite Smart Score

Each of six metrics is converted to a percentile rank across the candidate pool, then blended:

Smart Score

```
composite = 0.25·pct(Sharpe) + 0.15·[ pct(max_drawdown) + pct(profit_factor)
                                     + pct(ROI) + pct(PnL) + pct(volume) ]
```

```
confidence = clip( days_observed / 21 , 0.30 , 1.00 )
```

```
smart_score = composite × confidence → keep the top 100
```

Weighting is ~70% risk-adjusted skill & risk-control / 30% scale. Max drawdown is stored negative so a smaller drawdown ranks higher automatically. Confidence shrinks wallets with a short track record toward a neutral score rather than trusting a lucky two-week run at face value. The surviving top 100 become the tracked smart-money cohort — the single definitional filter that lives in the warehouse.

Why it matters: surfaces skilled, risk-controlled traders instead of lucky whales — a wallet with enormous PnL but a -60% drawdown ranks below a smaller account with a clean Sharpe. The strongest signal on the dashboard is convergence: when several independently top-scored wallets crowd into the same asset and direction, that is high-conviction smart money agreeing — not one account's gamble.

1.5 The Ten Charts and the Edge Each Gives

Chart	Reading it
BTC Market Regime (HMM)	Macro bias: trend-follow in Bull/Accumulation; de-risk as Bear/Capitulation probability rises.
Fear & Greed Index	Contrarian gauge: extreme fear often marks capitulation lows; extreme greed marks froth.
Hottest Coins (24h volume surge)	Where attention is flooding in; a volume spike often precedes a large move.
Strongest Coins (30d vs BTC)	Relative-strength leaders — momentum longs in an alt-friendly tape.
OI Dominance Treemap	Where leverage is concentrated (size) and flowing (colour) — squeeze-risk map.
Top OI Increases (7d)	Where fresh leveraged capital entered this week, paired with price direction.
Most Leveraged Coins (OI/mcap)	Perp market vs spot size — high ratio = crowded, liquidation-cascade-prone.
Smart-Money Leaderboard	The 100 highest-skill wallets ranked by risk-adjusted score, with live positions.
Net Positioning by Asset	Share of the cohort long/short/flat per asset — where conviction (and crowding) sits.
Smart-Money Flows (24h)	Day-over-day change in cohort net exposure — capital building or de-risking, by coin.

2 Feature 2 — Funding Rate Arbitrage Scanner

A delta-neutral carry screener. Perp exchanges charge a periodic funding rate to tether the perp price to spot; the same asset can carry very different funding on different venues. The trade: long the leg with cheap/negative funding and short the leg with expensive/positive funding on another exchange — collecting the funding difference while the two price exposures cancel, leaving (near) zero directional risk. The scanner finds spreads wide and, crucially, durable enough to be worth running.

2.1 Data Workflow

The top 20 assets by market cap (stablecoins excluded) are tracked across three exchanges. Funding intervals differ, so everything is normalized to a comparable 1-hour rate before comparison:

Exchange	Native funding	Handling
Hyperliquid	1-hour	Used as-is.
Aster	8-hour	Unit-normalized ($\div 8$) to a 1-hour-equivalent rate.
Lighter	8-hour	Unit-normalized ($\div 8$) to a 1-hour-equivalent rate.

Cadence & resilience: current rates refresh every 1 minute; the rolling 7-day (168-hour) hourly history that powers the statistics refreshes every 20 minutes. Aster's funding history is proxied through a lightweight endpoint to sidestep browser CORS restrictions. The spread series is simply shortLeg APY – longLeg APY, resampled to an hourly series over the trailing 7 days.

2.2 The Two Quality Metrics (the math)

A wide spread on a single snapshot is a trap — it may have existed for one hour, or flip sign constantly. Two statistics separate a real, harvestable edge from noise:

Persistence — has this opportunity actually kept paying?

$\text{persistence}(\text{window}) = \text{count}(\text{spread}_t > 0) / \text{count}(\text{window})$ (computed for 24h and 7d)

Sign-aware by design: the fraction of the window the spread stayed in the profitable direction. A spread that flips sign hourly scores low even if its average looks attractive.

Stability — how steady is the size of the edge? (the resilience measure)

$\text{stability} = \text{mean}(|\text{spread}|) / (\text{mean}(|\text{spread}|) + \text{stdev}(|\text{spread}|)) = 1 / (1 + \text{CV})$

→ 1 as the magnitude holds flat under volatility (resilient / dependable); → 0 as it spikes and collapses (noisy). This is the signal-to-noise core of a t-statistic applied to the spread's absolute size, independent of direction. CV = coefficient of variation = stdev / mean.

Ranking: opportunities are filtered to $\geq 3\%$ annualized spread and ranked by $\text{score} = \text{spreadAPY} \times (1 + \text{persistence7d})$ — rewarding wide spreads, but only after confirming they held up across the past week rather than at this instant alone.

Why it matters: separates genuinely exploitable carry trades from spreads that merely look wide on a stale snapshot. Persistence answers “has the direction been reliable,” Stability answers “is the size of the edge dependable” — together they filter out the flickering spreads that look great in a screenshot but vanish the moment real capital tries to capture them.

3 Feature 3 — DEX Trading Cost Comparison

Two panels. First, per-exchange fundamentals cards (TVL, Volume, Revenue — sourced from DefiLlama, toggleable across daily/weekly/monthly/quarterly/yearly) for sizing up each venue. Second, a live execution-cost calculator: pick an asset and a trade size, and it walks each exchange's real order book to show the true all-in cost of that specific market order — not just the advertised taker fee.

3.1 Workflow

Live Level-2 order books are pulled per exchange. A **VWAP book-walk** then consumes the ask ladder level by level until the requested notional is filled, returning the volume-weighted average fill price. If the book cannot absorb the size, it returns “insufficient depth” — itself an honest liquidity signal for thin pairs.

All-in execution cost breakdown

mid = (bestBid + bestAsk) / 2

spreadCost % = (bestAsk - mid) / mid

slippage % = (avgFillPrice - bestAsk) / mid (avgFillPrice from the VWAP book-walk)

totalCost % = takerFee % + spreadCost % + slippage %

Taker fees (static from exchange docs, except Lighter which is read live): Hyperliquid 0.045% · Aster 0.035% · Lighter 0% (currently promotional). Spread cost captures crossing half the bid-ask; slippage captures walking deeper into the book for larger size.

Why it matters: the venue with the lowest headline fee is frequently not the cheapest place to actually trade — for meaningful size, spread and slippage routinely dominate the fee entirely. This shows a trader the real, size-adjusted cheapest execution venue for their exact trade, and flags venues too thin to fill it at all.

4 Technology Stack & Engineering Themes

Layer	Tools
Frontend	React 19 · TypeScript · Vite · Tailwind CSS · MUI · Recharts
Data extraction	Python 3.11 · requests · pandas · numpy
Warehouse & transforms	Google BigQuery (free sandbox) · dbt Core (dbt-bigquery) · SQL
Machine learning	hmmlearn / scikit-learn / scipy (offline training) · hand-written TypeScript inference (runtime)
Orchestration & ops	GitHub Actions (daily cron + CI) · Google Cloud service accounts & secrets
Business intelligence	Power BI (native BigQuery connector, scheduled refresh)

Engineering themes a reviewer should notice

- **Analytics engineering done properly:** a layered dbt warehouse (raw → staging → marts), tested, documented with a lineage graph, and built on the principle that facts store the full population while presentation concerns (ranking, top-N) stay in the client.
- **Idempotent, reproducible pipeline:** partition-truncate loads mean re-runs never duplicate; raw retention means the entire history can be rebuilt from source without re-hitting APIs.
- **A real ML lifecycle:** unsupervised model selection with proper validation, then a train-offline / infer-in-the-browser split via exported parameters — the right pattern for a client-side app.
- **Quantitative rigor aimed at real users:** risk-adjusted (not raw-PnL) wallet scoring, sign-aware persistence and coefficient-of-variation stability for arbitrage, and true all-in execution cost rather than headline fees.
- **Cost-conscious architecture:** the entire pipeline runs at \$0 on free tiers — BigQuery sandbox, GitHub Actions minutes, and public data APIs — with credentials handled via encrypted secrets, never in code.

In one line: *PerpScope is a perpetual-DEX analytics platform that pairs a self-built, automated ELT pipeline and a trained market-regime model with live cross-exchange calculators — turning scattered raw exchange data into a clear macro read, a curated smart-money signal, and concrete execution decisions.*